Grokking Algorithms

Chapter 1: Introduction to Algorithms

- Binary Search (Tìm kiếm nhị phân) - cần ghi nhớ
  - Input (Đầu vào): Một danh sách đã được sắp xếp
  - Output (Đầu ra): Vị trí của phần tử cần tìm (index), hoặc null nếu không tìm thấy
  - Ứng dụng: Tìm kiếm trong danh sách đã sắp xếp
  - Hiệu quả hơn so với Simple Search (Tìm kiếm tuần tự)
- Big O Notation (Ký hiệu Big O) - cần ghi nhớ
  - Đo lường tốc độ tăng trưởng của thời gian chạy thuật toán (runtime) khi kích thước đầu vào (input size) tăng
  - Không đo lường tốc độ theo giây, mà dựa vào số lượng phép toán
  - Ví dụ:
    - O(log n) - Logarithmic time (Thời gian logarit)
    - O(n) - Linear time (Thời gian tuyến tính)
    - O(n log n) - Loglinear time
    - O(n^2) - Quadratic time (Thời gian bậc hai)
    - O(n!) - Factorial time (Thời gian giai thừa)

Chapter 2: Selection Sort

- How Memory Works (Bộ nhớ hoạt động như thế nào)
  - Bộ nhớ máy tính như một tủ ngăn kéo khổng lồ, mỗi ngăn kéo có một địa chỉ.
- Arrays and Linked Lists (Mảng và Danh sách liên kết) - cần ghi nhớ
  - Arrays (Mảng): Lưu trữ các phần tử liền kề nhau trong bộ nhớ
    - Ưu điểm: Truy cập ngẫu nhiên (random access) nhanh - O(1)
    - Nhược điểm: Chèn (insert) và xóa (delete) chậm - O(n)
  - Linked Lists (Danh sách liên kết): Các phần tử có thể ở bất kỳ đâu trong bộ nhớ, mỗi phần tử lưu trữ địa chỉ của phần tử tiếp theo (next element)
    - Ưu điểm: Chèn và xóa nhanh - O(1)
    - Nhược điểm: Truy cập ngẫu nhiên chậm - O(n)
- Selection Sort (Sắp xếp chọn) - tìm hiểu sau
  - Một thuật toán sắp xếp đơn giản
  - Độ phức tạp thời gian: O(n^2) - chậm

Chapter 3: Recursion (Đệ quy) - cần ghi nhớ

- Recursion: Khi một hàm gọi chính nó.
- Base Case (Trường hợp cơ sở): Khi hàm không gọi lại chính nó, ngăn chặn vòng lặp vô hạn (infinite loop).
- Recursive Case (Trường hợp đệ quy): Khi hàm gọi lại chính nó.
- The Stack (Ngăn xếp): Cấu trúc dữ liệu lưu trữ các lời gọi hàm (function call), hoạt động theo nguyên tắc LIFO (Last-In, First-Out) - vào sau ra trước.

Chapter 4: Quicksort - tìm hiểu sau

- Divide and Conquer (Chia để trị): Kỹ thuật chia bài toán thành các bài toán con nhỏ hơn và giải quyết chúng.
- Quicksort: Thuật toán sắp xếp hiệu quả sử dụng đệ quy.
- Big O Notation Revisited (Ôn lại Ký hiệu Big O)
  - Độ phức tạp thời gian trung bình (average case) của Quicksort là O(n log n)
  - Hằng số (constant) trong Big O thường không quan trọng.

Chapter 5: Hash Tables (Bảng băm) - cần ghi nhớ

- Hash Function (Hàm băm): Ánh xạ chuỗi (string) thành số (number).
- Hash Table: Cấu trúc dữ liệu lưu trữ các cặp key-value.
- Ứng dụng
  - Tra cứu (Lookups)
  - Ngăn chặn trùng lặp (Prevent duplicate entries)
  - Caching (Bộ nhớ đệm)
- Collisions (Va chạm): Khi hai key ánh xạ tới cùng một vị trí trong bảng băm.
- Performance (Hiệu suất)
  - Thời gian chạy trung bình: O(1) - rất nhanh
  - Load Factor (Hệ số tải): Số lượng phần tử được lưu trữ trên tổng số vị trí trong bảng băm.

Chapter 6: Breadth-First Search (Tìm kiếm theo chiều rộng) - cần ghi nhớ

- Graphs (Đồ thị): Mô hình hóa các kết nối giữa các đối tượng.
  - Nodes (Nút): Các đối tượng trong đồ thị.
  - Edges (Cạnh): Các kết nối giữa các nút.
- Breadth-First Search (BFS): Tìm kiếm đường đi ngắn nhất giữa hai nút trong đồ thị.
- Directed vs. Undirected Graphs (Đồ thị có hướng và đồ thị vô hướng)
- Queues (Hàng đợi): FIFO (First-in, First-out) - vào trước ra trước.

Chapter 7: Trees (Cây) - cần ghi nhớ

- Trees: Đồ thị không có chu trình (cycle).
- Depth-First Search (DFS) (Tìm kiếm theo chiều sâu): Duyệt qua tất cả các nút trong cây.
- Binary Trees (Cây nhị phân): Mỗi nút có tối đa hai nút con (child node).
- Huffman Coding (Mã hóa Huffman): Thuật toán nén sử dụng cây nhị phân.

Chapter 8: Balanced Trees (Cây cân bằng)

- Binary Search Trees (BSTs) (Cây tìm kiếm nhị phân): Cây nhị phân với tính chất đặc biệt để tìm kiếm hiệu quả.
- AVL Trees: Cây BST tự cân bằng.
- B-Trees: Cây BST tổng quát, thường được sử dụng trong cơ sở dữ liệu.

Chapter 9: Dijkstra's Algorithm (Thuật toán Dijkstra) - tìm hiểu sau

- Weighted Graphs (Đồ thị có trọng số): Mỗi cạnh có trọng số (weight) thể hiện chi phí di chuyển.
- Dijkstra's Algorithm: Tìm đường đi ngắn nhất trong đồ thị có trọng số.
- Negative-Weight Edges (Cạnh có trọng số âm): Thuật toán Dijkstra không hoạt động với cạnh có trọng số âm.

Chapter 10: Greedy Algorithms (Thuật toán tham lam)

- Greedy Strategy (Chiến lược tham lam): Chọn lựa chọn tối ưu cục bộ ở mỗi bước.
- The Knapsack Problem (Bài toán cái túi)
- The Set-Covering Problem (Bài toán phủ tập hợp)
- Approximation Algorithms (Thuật toán xấp xỉ): Tìm giải pháp gần đúng cho các bài toán NP-hard.

Chapter 11: Dynamic Programming (Quy hoạch động) - tìm hiểu sau

- Dynamic Programming: Chia bài toán thành các bài toán con nhỏ hơn, giải các bài toán con và lưu trữ kết quả để sử dụng lại.
- The Knapsack Problem Revisited (Ôn lại Bài toán cái túi)
- Longest Common Substring (Chuỗi con chung dài nhất)

Chapter 12: K-Nearest Neighbors (KNN) (K láng giềng gần nhất)

- KNN: Phân loại hoặc dự đoán dựa trên các điểm dữ liệu gần nhất.
- Feature Extraction (Trích xuất đặc trưng): Chuyển đổi dữ liệu thành các đặc trưng số (number).
- Regression (Hồi quy): Dự đoán một giá trị số.
- Machine Learning (Học máy): Giới thiệu về học máy.

Chapter 13: Where to Go Next (Các hướng tìm hiểu tiếp theo)

- Linear Regression (Hồi quy tuyến tính)
- Inverted Indexes (Chỉ mục đảo ngược)
- Fourier Transform (Biến đổi Fourier)
- Parallel Algorithms (Thuật toán song song)
- MapReduce
- Bloom Filters
- HyperLogLog
- HTTPS and Diffie-Hellman Key Exchange
- Locality-Sensitive Hashing
- Min Heaps and Priority Queues
- Linear Programming (Quy hoạch tuyến tính)